



Building a Radar-Equivalent Trust & Risk Platform

Why fraud decisioning belongs inside the payment decision plane, why fraud *intelligence* must live behind it, and how to keep the two apart without losing the latency budget.

Subject

Fraud & Risk
Architecture

Relates to

UPOP Decision
Engine

Parts

Two

Status

Working draft

Contents

This note compiles two connected discussions. The first establishes the overall shape of a Radar-equivalent capability: three risk planes, an attack-state service, the network effect as moat, and an economically optimal decision rather than a binary fraud verdict. The second goes a level deeper on the mechanics — what “precomputed” actually means, where the synchronous boundary sits, and which interfaces should and should not be HTTP.

On the Stripe references. Every claim attributed to Stripe below comes from its public engineering and product material. The original inline source links were not preserved through compilation, so these should be treated as directional characterisations to be re-verified before they appear in any external or investment-facing document. The latency budgets, p-tier targets and interface contracts are architecture recommendations, not observed Stripe internals.

PART I — THE SHAPE OF THE PLATFORM

1. The key architectural principle	4
2. What Radar-like risk is actually trying to detect	6
3. Your architecture should have three risk planes	7
4. The hot path	8
5. The trick — don't query the fraud graph synchronously	8
6. Card testing is particularly suited to streaming intelligence	10
7. I would introduce an “Attack State” service	11
8. This is where the network effect becomes the moat	11
9. But don't make the network model the only model	12
10. Now connect this to your Decision Engine	13
11. But I would keep Risk physically separated	14
12. The architecture I'd recommend for your PSP	16
13. The most important latency architecture	17
14. And this fits with your experimentation vision	18
15. The really interesting opportunity for your PSP	19

Recommended final architecture	19
--------------------------------	----

PART II – PRECOMPUTATION, BOUNDARIES AND INTERFACES

1. Is Risk ↔ Payment over HTTP?	22
2. But don't confuse Risk Decision with Risk Data	23
3. What exactly does “precomputed” mean?	23
4. Better architecture — continuously maintain the answer	24
5. Think of it like a scoreboard	25
6. The payment request then becomes very small	25
7. But some features are calculated during the payment	27
8. Now let's walk through card testing	28
9. There is another important piece — graph intelligence	29
10. What happens after the payment?	30
11. Where HTTP does make sense	31
12. The architecture I would recommend for your PSP	31
13. The critical architectural boundary	33
14. One more thing I would change in your architecture	33
The simplest way to remember it	34

The Shape of the Platform

Three risk planes, an attack-state service, and the network as the moat.

Fraud detection cannot simply be another synchronous microservice sitting in the authorization path. This is one of the most important architectural questions for the PSP being designed.

Based on Stripe's public engineering material, I would model Radar-like capabilities as a real-time risk decision plane tightly integrated with payment processing, but architecturally separated from the payment execution plane. Stripe says Radar evaluates transactions in real time, uses hundreds or thousands of signals, and has described its decision as taking under 100 ms.

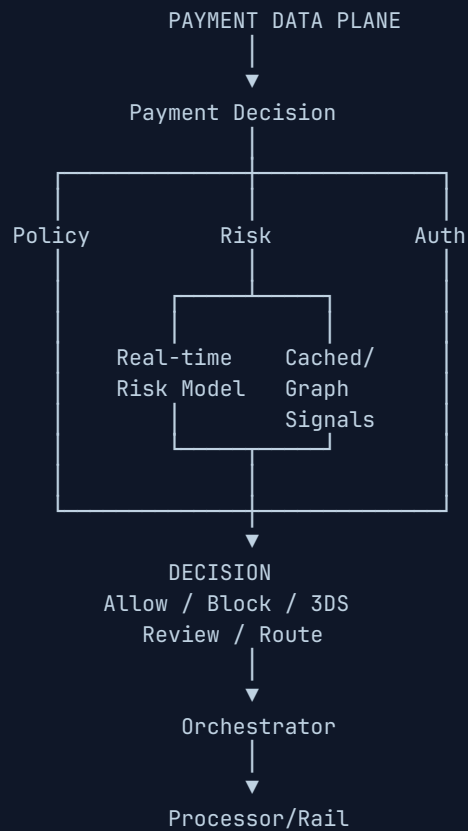
I would design your version like this.

1. The key architectural principle

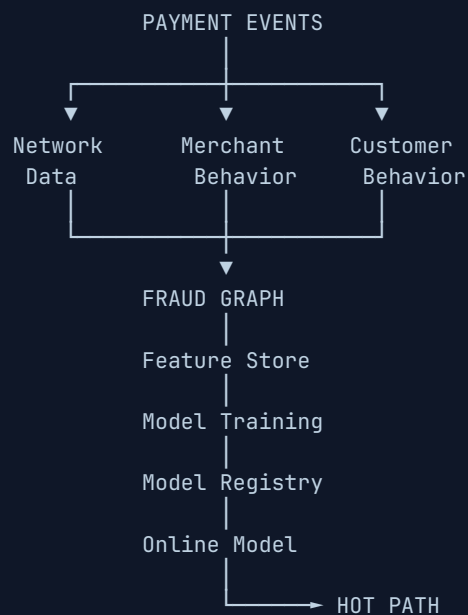
DON'T BUILD THIS

```
Payment Gateway
  ↓
Call Fraud Service
  ↓
Wait 200 ms
  ↓
Continue Payment
```

BUILD THIS INSTEAD



Meanwhile, behind it:



The hot path consumes precomputed intelligence. It does not build the intelligence from scratch. That is the fundamental latency architecture.

2. What Radar-like risk is actually trying to detect

You mentioned three categories, and I would explicitly separate them.

A. CARD TESTING

The attacker is trying to determine: “Which stolen or generated cards are valid?”

Typical pattern:

```
1 IP/device
├── Card A → decline
├── Card B → decline
├── Card C → approve
├── Card D → decline
├── Card E → approve
└── ...
```

Velocity, card/IP/device relationships and behavioral patterns become extremely important.

Stripe has publicly discussed card-testing detection using rate limiters, alerts, models and challenges; its Checkout CAPTCHA intervention reportedly reduced card testing by 80% with less than 2 bps authorization-rate impact.

More recently, Stripe described moving beyond simple low-authorization-rate enumeration signals toward probabilistic models for compromised cards.

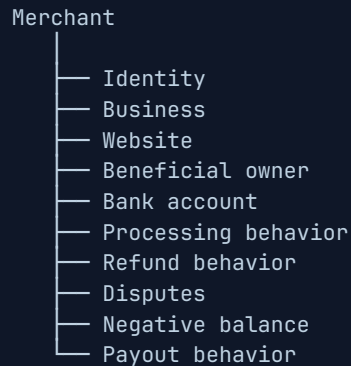
B. CONSUMER / PAYMENT FRAUD

```
Real customer account
┆
┆ ▼
┆ Stolen card
┆   ┆
┆   ┆ ▼
┆   ┆ $4,000 electronics
┆   ┆   ┆
┆   ┆   ┆ ├── New device
┆   ┆   ┆ ├── New shipping address
┆   ┆   ┆ ├── New IP
┆   ┆   ┆ ├── Unusual velocity
┆   ┆   ┆ └── Different geography
```

This requires transaction-level and customer-level risk.

C. MERCHANT / PLATFORM FRAUD

This is fundamentally different. You're asking: *"Should this merchant or account be trusted with payment acceptance and eventually with money?"*

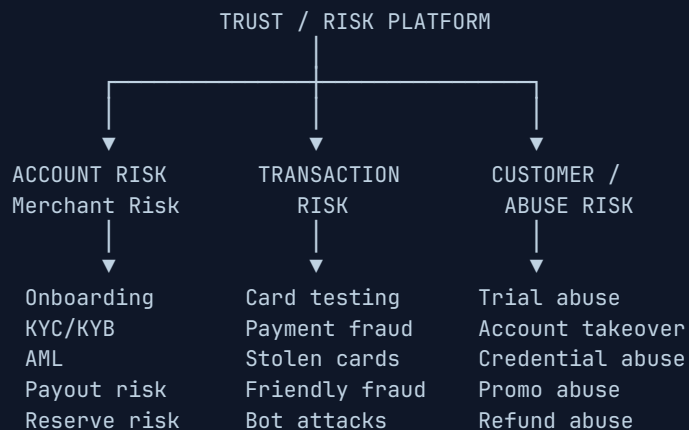


Stripe's current Radar positioning explicitly extends beyond transaction fraud into account/merchant risk, onboarding, payouts and customer abuse.

I would not put merchant fraud and payment fraud into one model.

3. Your architecture should have three risk planes

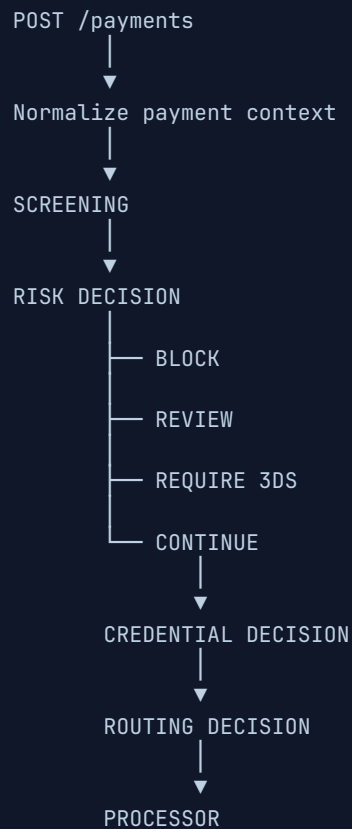
I would make this explicit in your PSP architecture.



Then the payment path consumes their decisions.

4. The hot path

Let's take a normal \$500 ecommerce payment.



The risk engine should have a strict latency budget. For example, if your overall target is under 300 ms:

API / normalization	10 ms
Policy + screening	10 ms
Risk decision	20-40 ms
Credential decision	5-10 ms
Routing	5-10 ms
Processor/network	remaining

These are architecture targets, not claims about Stripe's internal timings. Stripe publicly says Radar itself can make its decision in under 100 ms.

5. The trick — don't query the fraud graph synchronously

This is where I think your architecture can be significantly better than the typical “fraud microservice” design.

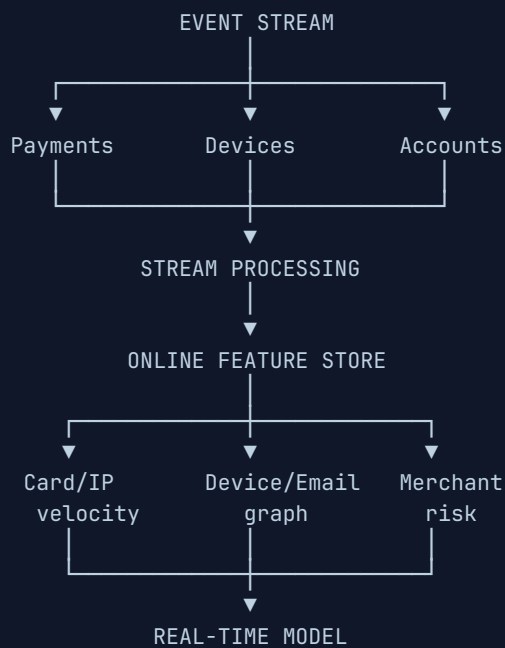
Imagine the payment arrives:

```
Card = X
IP = Y
Device = Z
Email = A
Merchant = M
Amount = $500
```

You don't want:

```
Risk Engine
↓
Redis
↓
Customer DB
↓
Device DB
↓
Card DB
↓
Merchant DB
↓
Network DB
↓
ML service
```

That creates unpredictable tail latency. Instead, maintain an online fraud feature store:



Then the payment asks: *“Give me the precomputed risk features for Card X + IP Y + Device Z + Merchant M.”*
That’s dramatically faster.

6. Card testing is particularly suited to streaming intelligence

Suppose an attacker suddenly sends:

```
10:00:00  1 attempt
10:00:01  3 attempts
10:00:02  12 attempts
10:00:03  50 attempts
10:00:04  200 attempts
```

You don't want the next transaction waiting for an offline fraud model. A streaming system continuously computes:

```
IP velocity
Device velocity
Card velocity
Email velocity
BIN velocity
Merchant velocity
Decline velocity
Approval velocity
Amount distribution
Geographic dispersion
Card-to-device relationships
Device-to-account relationships
```

Then the risk engine sees:

```
IP risk = HIGH
Device risk = HIGH
Merchant attack state = ACTIVE
Card risk = UNKNOWN
```

and immediately changes behavior. For example:

```
IF attack_state = ACTIVE
AND device_velocity > threshold
THEN
    BLOCK
```

or:

```
IF attack_state = ACTIVE
AND risk = MEDIUM
THEN
    REQUIRE_3DS / CHALLENGE
```

Stripe has publicly described dynamic risk thresholds that become more aggressive when an attack is detected and then relax after the attack subsides. That’s an important concept for your architecture.

7. I would introduce an “Attack State” service

This is something I’d add to your PSP design. Instead of risk only answering `risk_score = 72` , have:

```
Threat Intelligence
  |
  v
Attack State
```

Examples:

```
NORMAL
CARD_TESTING_ATTACK
DEVICE_FARM_ATTACK
ACCOUNT_TAKEOVER_ATTACK
PROMO_ABUSE_ATTACK
BOT_ATTACK
MERCHANT_FRAUD_ATTACK
```

Then your decision engine incorporates the state. A payment risk of 42 might normally be acceptable. But:

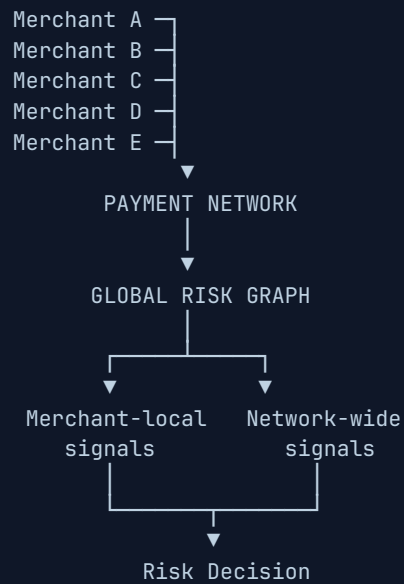
```
Merchant Attack State = CARD_TESTING
Device Attack State   = ACTIVE
```

could cause `BLOCK` even though the individual payment looks relatively normal.

That is collective intelligence, rather than transaction scoring.

8. This is where the network effect becomes the moat

Stripe publicly says Radar uses signals across its network and that a large percentage of cards have been seen previously on Stripe. Your equivalent PSP architecture should therefore have:



That allows you to detect: *“This device has never attacked Merchant A, but it attacked 14 other merchants yesterday.”*

Merchant A couldn’t see that by itself. The PSP can.

9. But don’t make the network model the only model

You need three layers.

LAYER 1 — MERCHANT-SPECIFIC

Merchant rules
Customer history
Merchant risk appetite

LAYER 2 — PSP NETWORK

Card
Device
IP
Email
Identity
BIN
Merchant relationships
Cross-merchant behavior

LAYER 3 — EXTERNAL INTELLIGENCE

```
Network alerts
Issuer signals
TC40/SAFE
Early fraud warnings
Threat intelligence
Sanctions/watchlists
```

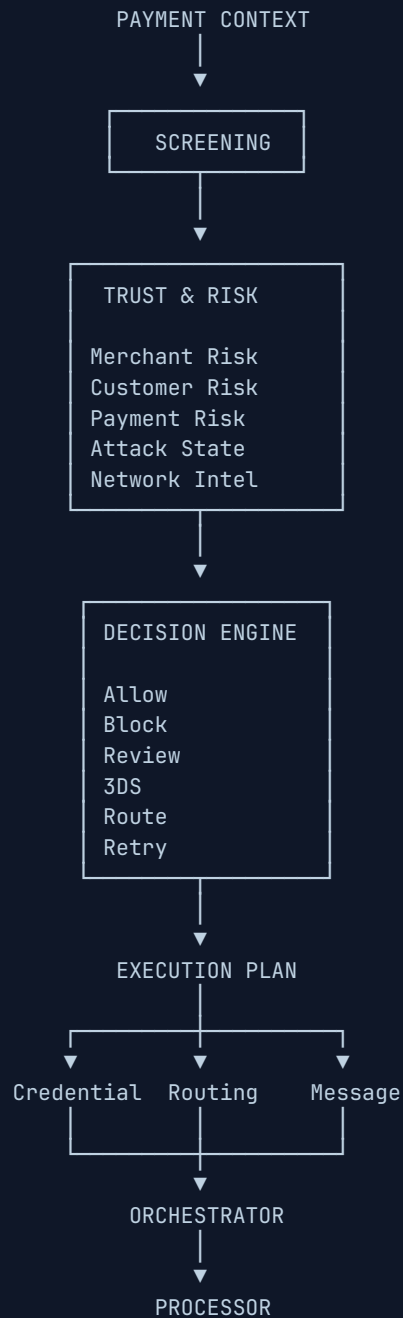
Stripe publicly describes ingesting payment, checkout, card-network and bank information and references network fraud signals such as TC40s, SAFE reports and early dispute notifications.

10. Now connect this to your Decision Engine

This is where I'd change your UPOP architecture slightly. You currently have:

```
Policy
Screening
Risk
Authentication
Credential
Message
Routing
Retry
```

I would evolve it to:



This is much more powerful than “Risk calls Radar.” Risk becomes a decisioning capability inside the Payment Decision Engine.

11. But I would keep Risk physically separated

This distinction is subtle but important.

LOGICAL ARCHITECTURE

Risk is part of the **Payment Decision Engine**.

PHYSICAL ARCHITECTURE

Risk is an independent platform: the **Trust & Risk Platform**.

Why? Because the same risk intelligence is needed for:

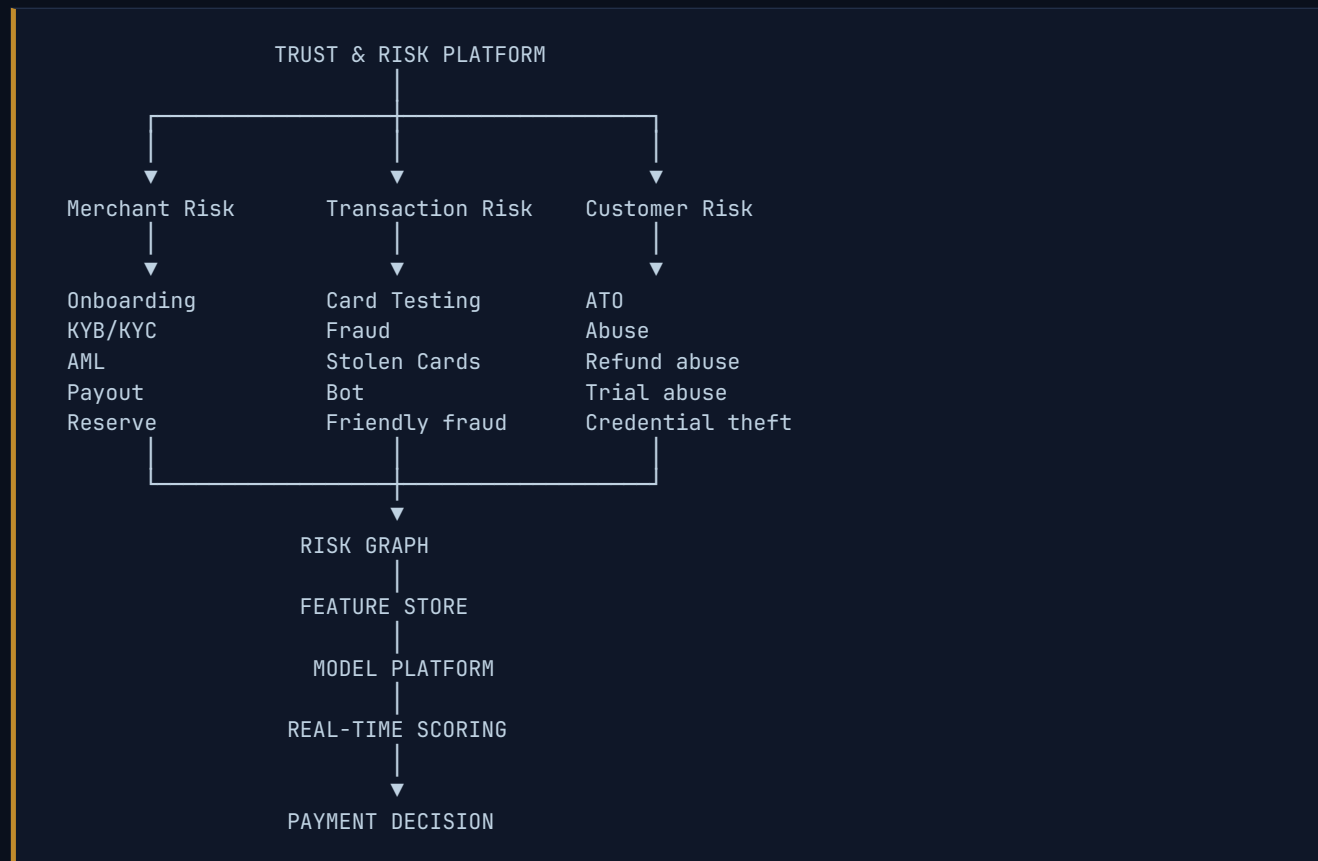
```
graph TD; A[Merchant onboarding] --> B[Payment]; B --> C[Account changes]; C --> D[Subscription]; D --> E[Refund]; E --> F[Payout]; F --> G[Dispute];
```

Merchant onboarding
↓
Payment
↓
Account changes
↓
Subscription
↓
Refund
↓
Payout
↓
Dispute

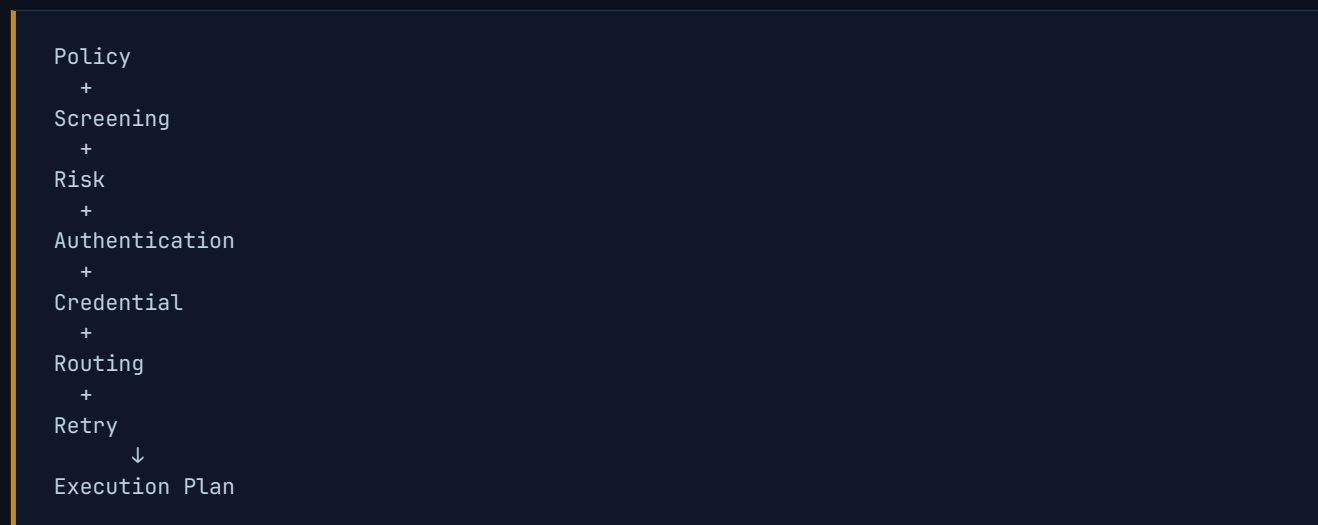
You don't want fraud intelligence trapped inside the card authorization service.

12. The architecture I'd recommend for your PSP

TRUST & RISK PLATFORM



AND SEPARATELY — THE DECISION PLANE

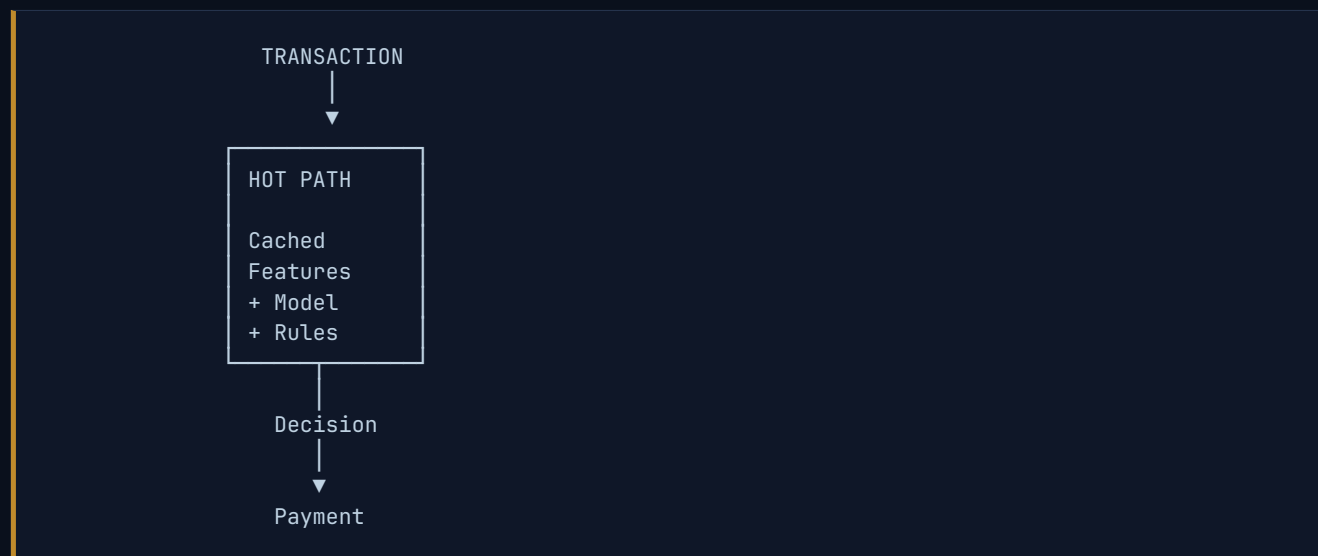


13. The most important latency architecture

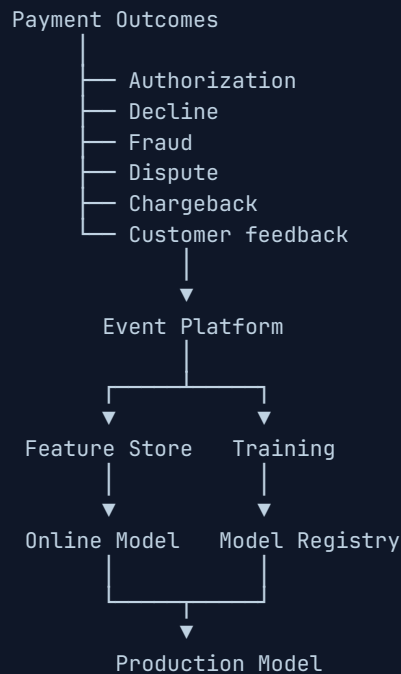
I'd establish three classes of intelligence.

INTELLIGENCE	EXAMPLE	PATH
Synchronous	Risk score	~50 ms target
Near-real-time	Attack state / velocity	Continuously updated
Asynchronous	Model training / dispute outcomes	Event-driven

So:



while:



This gives you continuous learning without putting model training anywhere near the payment latency path.

14. And this fits with your experimentation vision

You previously wanted your PSP to be able to say: *“Deploy a hypothesis, test it, measure it, and promote it.”* Fraud should use the same platform.

```

Hypothesis:
"Device velocity > 15/minute is predictive of card testing."
↓
Backtest
↓
Shadow
↓
Canary
↓
Live Experiment
↓
Measure:
Fraud reduction
False positives
Authorization impact
Conversion impact
↓
PROMOTE MODEL
  
```

That becomes a Fraud Strategy Marketplace inside your broader Decision Platform.

15. The really interesting opportunity for your PSP

I would actually go one step beyond Radar's conceptual model. Don't optimize *"Is this transaction fraudulent?"*
Optimize:

What is the economically optimal action given fraud probability, approval probability, customer value and cost?

For example:

```
Payment = $800  
Fraud probability = 2%  
Approval probability = 94%  
Customer LTV = $25,000  
3DS cost/friction = high  
Chargeback cost = $100
```

Maybe **ALLOW**. Another transaction:

```
Fraud probability = 65%
```

→ **BLOCK**. Another:

```
Fraud probability = 8%  
High-value customer  
Strong device reputation  
3DS available
```

→ **3DS**. Another:

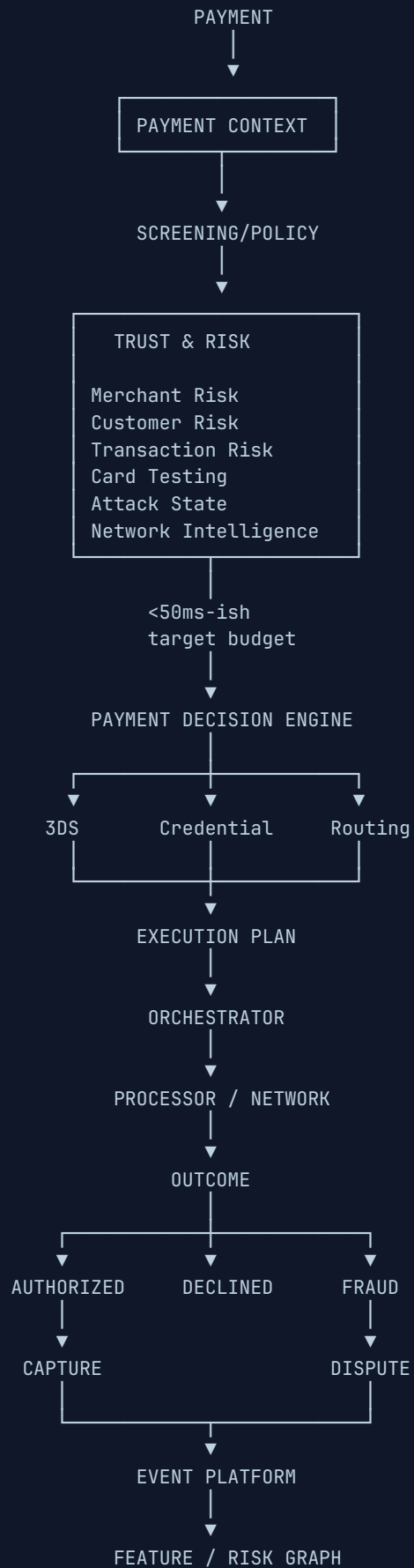
```
Fraud probability = 1%  
But processor has poor economics
```

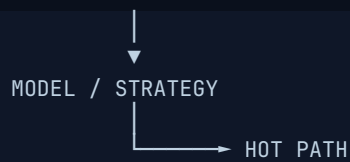
→ **ALLOW + OPTIMIZE ROUTE**

That is exactly where your Economic Governor + Decision Engine concept becomes differentiated.

Recommended final architecture

I would therefore make the relationship:





Don't build "Radar beside the gateway." Build a Trust & Risk intelligence platform behind the gateway, with an ultra-low-latency risk decision interface embedded in the Payment Decision Plane.

That lets the same intelligence protect merchant onboarding, card testing, consumer fraud, account takeover, payment fraud, refunds, disputes, payouts and platform/PayFac risk, while keeping the payment execution path fast. Stripe's current public positioning similarly spans transaction, account and customer-abuse risk, rather than treating fraud as only an authorization-time card score.

Precomputation, Boundaries and Interfaces

What is computed before the payment arrives, what is computed during it, and where the synchronous line is drawn.

“Precomputed” does not mean that the fraud decision is calculated before the payment arrives. It means that much of the state used to make the decision is continuously calculated before the payment arrives, so the online decision doesn’t have to reconstruct history from scratch.

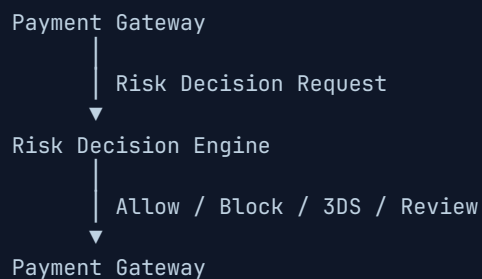
Stripe publicly describes exactly this type of architecture: Radar computes features in real time, maintains up-to-date state such as IP/card relationships, and makes the fraud decision in under 100 ms.

1. Is Risk ↔ Payment over HTTP?

It can be, but I would not make synchronous HTTP the fundamental architecture. There are really two different interactions.

A. PAYMENT → RISK — SYNCHRONOUS DECISION

The payment hot path needs an immediate answer:



This could technically be HTTP/REST, but for an internal PSP platform I would prefer gRPC over HTTP/2, or another very-low-latency RPC mechanism.

Why? Because this is a request/response decision, not a normal business API call. You want:

```
Risk.evaluate(payment_context)
      ↓
Decision
```

with predictable latency and tight timeouts. I would target something like:

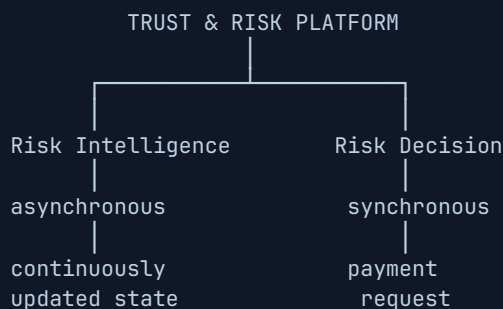
```
p50:    <10 ms
p95:    <25 ms
p99:    <50 ms
```

for the risk decision service itself, leaving the rest of the payment latency budget for authentication, routing, processor calls, etc.

These are architecture targets I'm recommending, not claims about Stripe's internal numbers. Stripe publicly says Radar makes its decision in under 100 ms.

2. But don't confuse Risk Decision with Risk Data

This is the most important distinction.



The payment does not call 15 databases to assemble fraud information. Instead, the Risk Decision Engine reads from low-latency online state.

3. What exactly does “precomputed” mean?

Let's take: Card X is being used from IP address 1.2.3.4.

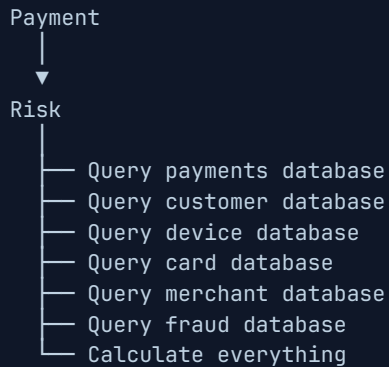
The risk model might care about:

- How many attempts in the last 1 minute?
- How many attempts in the last hour?
- How many were declined?

- How many different merchants?
- How many different countries?
- Has this device been associated with fraud?
- Has this card appeared on the network?

BAD ARCHITECTURE

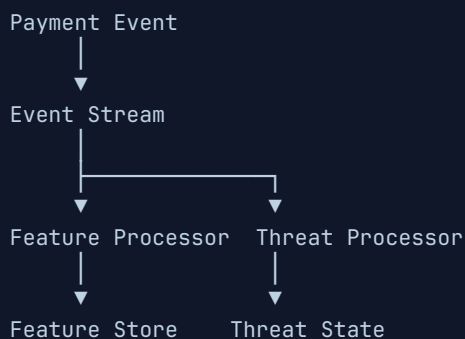
When the payment arrives:



That could easily destroy your latency and create unpredictable p99 performance.

4. Better architecture — continuously maintain the answer

Instead, every payment generates an event:



The feature processor might maintain:

```
IP = 1.2.3.4
cards_seen_1m      = 27
cards_seen_1h      = 143
attempts_1m        = 51
declines_1m        = 42
merchants_1h       = 8
countries_1h       = 4
fraud_associations = HIGH
```

Those values are precomputed state. Not the final fraud decision.

5. Think of it like a scoreboard

This analogy is useful. You don't want to calculate an NFL team's entire season record every time someone asks *"What's their record?"* You continuously maintain:

```
wins
losses
points
recent form
home record
away record
```

Then when somebody asks, the answer is immediate.

Fraud features work similarly. Instead of calculating *"How many cards has this IP touched in the last hour?"* from billions of transactions, you maintain that counter continuously.

6. The payment request then becomes very small

When the next transaction arrives:

```

Payment
{
  merchant_id,
  payment_id,
  amount,
  currency,
  card_token,
  customer_id,
  device_id,
  ip,
  email,
  shipping_address
}

```

Risk asks its online feature layer for a feature lookup, and receives something like:

```

{
  ip_velocity_1m: 51,
  ip_unique_cards_1h: 143,
  device_velocity_1m: 12,
  card_seen_before: true,
  device_risk: 0.71,
  email_risk: 0.12,
  merchant_attack_state: "CARD_TESTING",
  customer_history_score: 0.08
}

```

Then the ML model evaluates:

```

Payment Context
  +
Real-time Features
  +
Network Intelligence
  +
Merchant Rules
  +
Model
  ↓
Risk Score

```

This can happen very quickly. Stripe publicly says its system evaluates hundreds or thousands of signals and uses network-wide data, and its engineering documentation describes maintaining current feature state so those values can be accessed with low latency during the Stripe API flow.

7. But some features are calculated during the payment

This is an important nuance. “Precomputed” doesn’t mean everything is precomputed. I’d divide features into three categories.

TYPE 1 — STATIC / SLOWLY CHANGING

```
BIN country  
Card brand  
Merchant country  
Customer age of account  
Device reputation  
Card previously seen
```

These can be cached aggressively.

TYPE 2 — STREAMING / CONTINUOUSLY MAINTAINED

```
Cards per IP in last 1 minute  
Transactions per device in last 5 minutes  
Decline rate  
Unique customers per IP  
Unique cards per merchant  
Velocity  
Cross-merchant relationships
```

These are continuously updated as transactions occur. This is the heart of card-testing detection.

TYPE 3 — TRANSACTION-SPECIFIC

```
Amount  
Current IP  
Current device  
Current shipping address  
Current order  
Current payment method  
Current merchant
```

These obviously don’t exist until the payment arrives. So:

```
Risk Input
=
Current Transaction
+
Pre-existing Features
+
Real-time Streaming Features
+
Network Intelligence
```

8. Now let's walk through card testing

Suppose a fraudster has 500 stolen cards. They start attacking Merchant A.

TRANSACTION #1

```
IP = 10.1.1.1
Card = A

Feature state:
IP unique cards = 1

Risk: LOW
```

Maybe allowed.

TRANSACTION #10

```
IP unique cards = 10
attempts/minute = 10
```

Still perhaps ambiguous.

TRANSACTION #50

```
IP unique cards = 47
attempts/minute = 50
decline rate = 92%
```

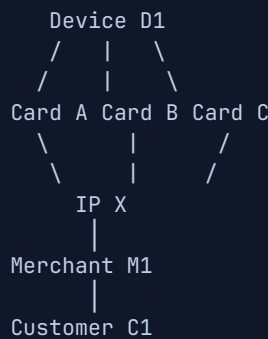
Now the streaming feature system has already established that this IP is highly suspicious. The next transaction does not need to discover that fact. It simply reads `ip_attack_score = 0.97`.

```
Payment
+
IP attack score
+
Device reputation
+
Card reputation
+
Merchant attack state
↓
BLOCK
```

This is what I mean by precomputed intelligence. Stripe’s public documentation specifically describes related-payment patterns such as many different cards sharing one IP as a way to identify card testing.

9. There is another important piece — graph intelligence

This gets more interesting. You don’t just maintain counters. You maintain relationships.



Then:

```
Card A → Device D1
Card B → Device D1
Card C → Device D1
Card D → Device D1
```

becomes a very strong signal. The Risk Engine can ask “*What do we know about this device/card/IP relationship?*” rather than “*Search the entire payment history.*”

Stripe publicly describes network-related payment relationships and signals such as cards, IP addresses and customer associations.

10. What happens after the payment?

This is equally important. Suppose:

```
Payment
  ↓
AUTHORIZED
  ↓
CAPTURED
```

Weeks later: a dispute, or confirmed fraud. That becomes a new event:

```
Fraud Outcome
  |
  ▼
Event Stream
  |
  ├── Feature Store
  ├── Risk Graph
  ├── Model Training
  ├── Rules Analytics
  └── Merchant Risk
```

Now future transactions benefit. That's your learning loop:

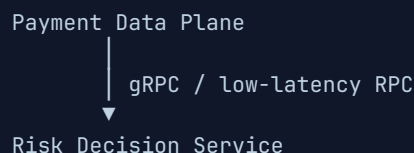
```
TRANSACTION
  ↓
RISK DECISION
  ↓
PAYMENT OUTCOME
  ↓
FRAUD / DISPUTE SIGNAL
  ↓
FEATURE UPDATE
  ↓
MODEL TRAINING
  ↓
NEW MODEL
  ↓
PRODUCTION
  ↓
BETTER RISK DECISION
```

This is the fraud intelligence flywheel.

11. Where HTTP does make sense

There are actually several interfaces.

REAL-TIME PAYMENT DECISION



RISK ADMINISTRATION

A normal API:

```
POST /risk/rules
GET /risk/decisions/{id}
POST /risk/models
```

HTTP/REST is perfectly appropriate here.

EVENT INGESTION

Don't use synchronous HTTP between every component. Use Kafka, Pulsar or an equivalent event bus — conceptually:

```
Payment → Event Stream → Risk Feature Pipeline
```

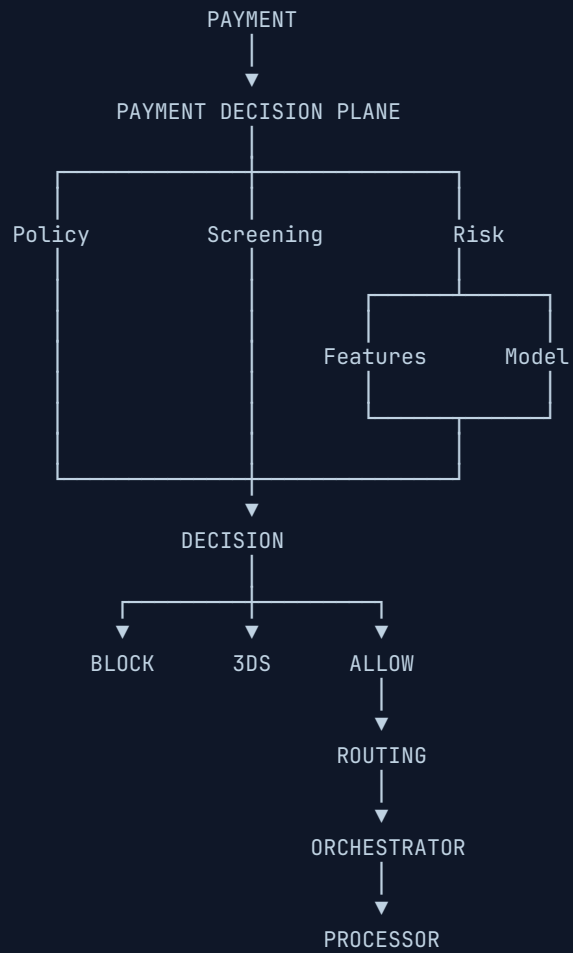
MODEL DEPLOYMENT

```
Model Registry
  ↓
Model Deployment
  ↓
Risk Runtime
```

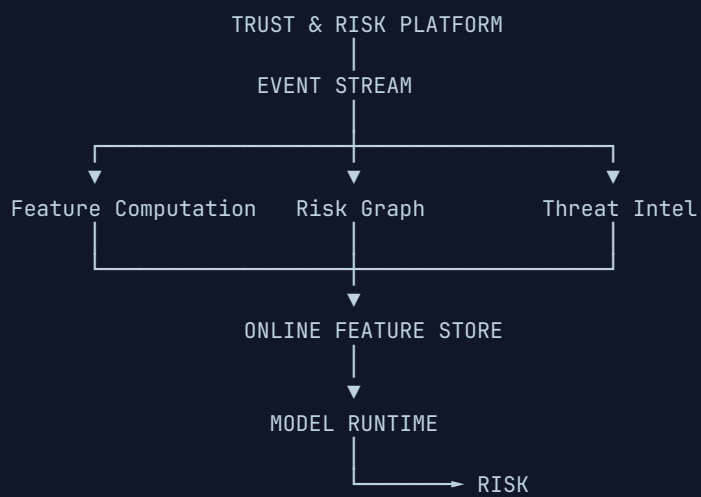
This is not in the payment hot path.

12. The architecture I would recommend for your PSP

I'd make the separation explicit:



Behind it:



13. The critical architectural boundary

I'd define the Risk Decision API as something like:

```
evaluatePaymentRisk(  
    PaymentContext,  
    RiskContext  
)  
    ↓  
RiskDecision
```

Response:

```
{  
  decision: ALLOW | BLOCK | CHALLENGE | REVIEW,  
  risk_score: 0.82,  
  risk_level: HIGH,  
  reason_codes: [...],  
  model_version: "...",  
  policy_version: "...",  
  decision_id: "..."  
}
```

Don't send the entire transaction history. Send the current payment context, while Risk retrieves its low-latency intelligence.

14. One more thing I would change in your architecture

Given your goal of sub-300 ms global PSP latency, I would actually make the Risk Engine part of your Decision Plane, but make its data dependencies asynchronous. That's the key.

SYNCHRONOUS

```
Payment  
  ↓  
Risk Decision  
  ↓  
Execution Plan
```

ASYNCHRONOUS

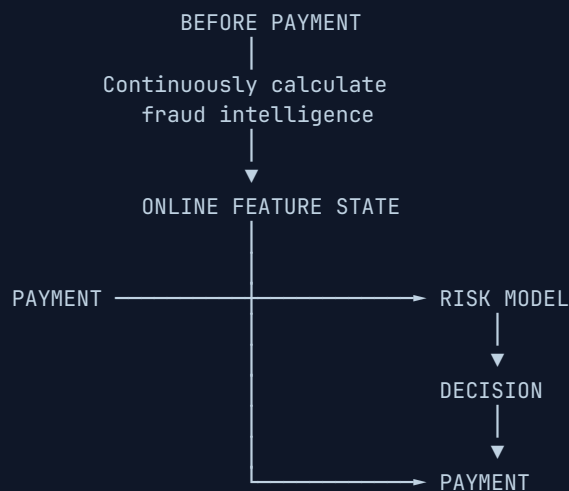
```
Every payment
↓
Event Stream
↓
Feature computation
↓
Risk graph
↓
Model learning
```

So you're not choosing between "Risk must be synchronous" and "Risk must be asynchronous." You need both.

Decision = synchronous. Intelligence generation = continuously asynchronous.

That is the architecture I would use to build your Radar-equivalent capability. And importantly, Stripe's public engineering material supports this general pattern: Radar performs the decision in the payment flow, while its ML infrastructure maintains current feature values in production and uses the broader network to generate predictive signals.

The simplest way to remember it



You don't precompute the answer. You precompute the evidence needed to answer the question quickly.

